16

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

DTO와 엔티티

제2부 NestJS 핵심 · 4장 구성 요소

받는 모양, 가진 모양, 보여 주는 모양



© 한상훈 NestJS 백엔드 마스터

공연 목록 응답

터미널

```
$ curl localhost:4016/concerts
[{"id":1,"settlementRate":0.1,"createdAt":"2026-10-26T04:59:11.724Z","title":"가을 콘서트","venue":"올림픽홀"}]
```

→ `settlementRate`: 주최사와 맺은 정산 수수료율

→ 고객 앱이 받는 JSON에 포함 (로그인 없는 API, 개발자 도구로 누구나 확인)

→ 처음에는 엔티티와 응답이 같은 모양: 나중에 붙인 필드가 응답으로

© 한상훈 NestJS 백엔드 마스터

가게 장부와 영수증

→ 가게 장부: 가게 안에서만 보는 종이, 원가·남는 돈·월세

→ 영수증: 고객이 받는 종이, 메뉴·수량·가격

→ 장부 줄을 영수증으로 그대로 주면: 고객이 원가와 가게가 남기는 돈을 봄



© 한상훈 NestJS 백엔드 마스터

Concert 엔티티

concert.entity.ts

```
export class Concert {
  id: number;
  title: string;
  venue: string;
  settlementRate: number;
  createdAt: Date;
}
```

→ 공개해도 되는 것: `id`, `title`, `venue`

→ 내부용: `settlementRate`(정산 수수료율), `createdAt`(등록 시각)

© 한상훈 NestJS 백엔드 마스터

DTO

→ DTO(Data Transfer Object): 경계를 넘어 데이터를 옮길 때의 모양만 적은 클래스

→ 요청 DTO: 클라이언트가 보내도 되는 필드

→ 응답 DTO: 클라이언트에게 보여 줘도 되는 필드

→ 메서드 없음, 필드 이름과 타입만

→ interface 대신 클래스 (Nest 문서 권장): 실행 코드에 남아 검사 규칙을 붙일 수 있음

© 한상훈 NestJS 백엔드 마스터

엔티티

→ 엔티티(entity): 서버가 업무 규칙과 함께 관리하는 데이터

→ id로 구분, 시간이 지나며 상태 변경

→ 내부 필드 포함: 수수료율, 등록 시각, 좌석 상태

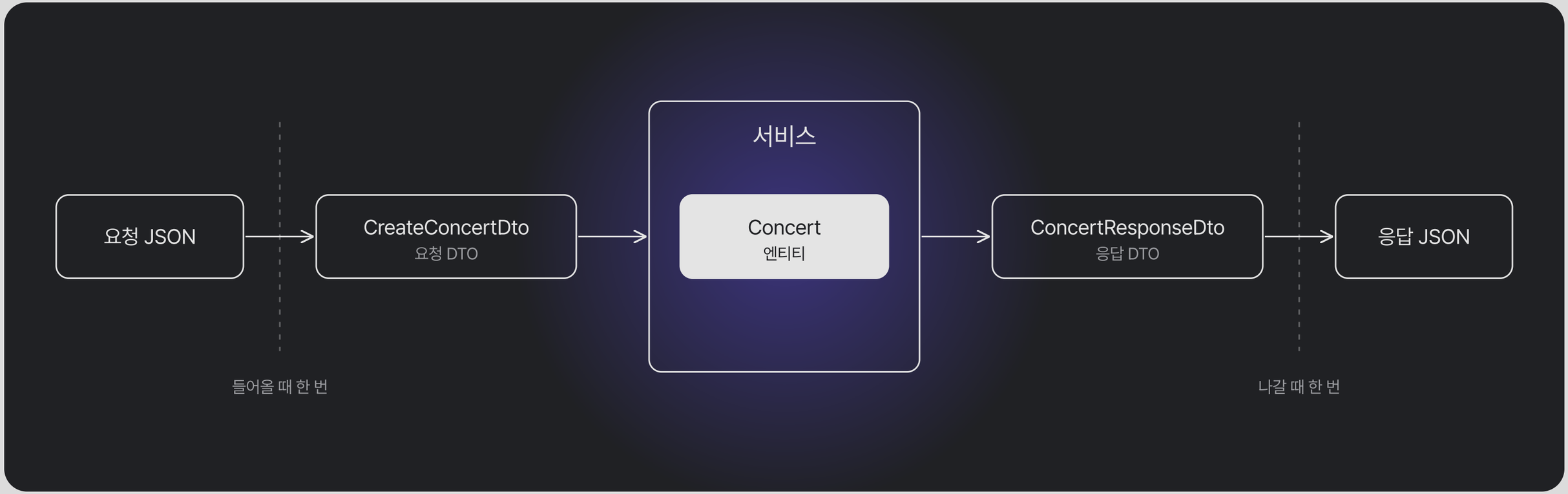
→ DB를 붙이면: 테이블 한 줄과 대응

© 한상훈 NestJS 백엔드 마스터

세 가지 모양

→ 요청 JSON → 요청 DTO(`CreateConcertDto`) → 서비스 → 엔티티(`Concert`)

→ 엔티티 → 응답 DTO(`ConcertResponseDto`) → 응답 JSON



© 한상훈 NestJS 백엔드 마스터

스키매틱이 나눠 둔 폴더

```
concerts/
├─── dto/
│    ├─── create-concert.dto.ts
│    └─── update-concert.dto.ts
└─── entities/
     └─── concert.entity.ts
```

→ `nest g resource`가 처음부터 dto와 entities를 따로 생성

© 한상훈 NestJS 백엔드 마스터

엔티티를 그대로 돌려주는 컨트롤러

concerts.controller.ts

```
  @Get()
  findAll() {
    return this.concertsService.findAll();   // Concert[]
  }
```

→ 엔티티 필드 전부 JSON으로

→ 엔티티에 필드를 더하면: 응답에도 자동 추가

© 한상훈 NestJS 백엔드 마스터

ConcertResponseDto

concert-response.dto.ts

```ts
import { Concert } from '../entities/concert.entity.js';

export class ConcertResponseDto {
  id: number;
  title: string;
  venue: string;

  static from(concert: Concert): ConcertResponseDto {
    return { id: concert.id, title: concert.title, venue: concert.venue };
  }
}
```

→ 보여 줄 필드를 하나씩 적어서 옮김

© 한상훈 NestJS 백엔드 마스터

응답 DTO로 바꾸는 컨트롤러

concerts.controller.ts

```
  @Post()
  create(@Body() createConcertDto: CreateConcertDto) {
    return ConcertResponseDto.from(this.concertsService.create(createConcertDto));
  }

  @Get()
  findAll() {
    return this.concertsService.findAll().map(ConcertResponseDto.from);
  }

  @Get(':id')
  findOne(@Param('id', ParseIntPipe) id: number) {
    return ConcertResponseDto.from(this.concertsService.findOne(id));
  }
```

© 한상훈 NestJS 백엔드 마스터

고친 응답

터미널

```
$ curl localhost:4016/concerts
[{"id":1,"title":"가을 콘서트","venue":"올림픽홀"}]
```

→ 엔티티에 내부 필드를 더해도 응답은 그대로

→ 목록용·상세용 응답 DTO 분리 가능 (예: 상세에만 소개글, 출연진)

→ 응답에 새 필드: 응답 DTO 파일 변경 → diff에서 공개 여부 검토

© 한상훈 NestJS 백엔드 마스터

서비스는 엔티티, 컨트롤러는 응답 DTO

→ 서비스 반환: 엔티티 (다른 서비스도 사용)

→ 예: PerformancesService가 `ConcertsService.findOne`으로 공연 확인

→ 응답 모양: HTTP 쪽 일, 컨트롤러에서 변환

→ 수수료율이 필요한 정산 기능: 엔티티를 받아 계산

© 한상훈 NestJS 백엔드 마스터

본문을 그대로 붓는 create

concerts.service.ts

```ts
  create(createConcertDto: CreateConcertDto) {
    const concert = {
      id: this.nextId++,
      settlementRate: 0.1,
      createdAt: new Date(),
      ...createConcertDto,
    } as Concert;
    this.concerts.push(concert);
    return concert;
  }
```

→ 의도: 기본값을 먼저 두고 요청 필드를 덮어씀

→ 실제: 요청에 온 필드 전부가 덮어씀

© 한상훈 NestJS 백엔드 마스터

본문에 끼워 넣은 필드

터미널

```
$ curl -X POST localhost:4016/concerts -d '{"title":"가을 콘서트","venue":"올림픽홀"}'
{"id":1,"settlementRate":0.1,"createdAt":"2026-10-26T04:59:11.724Z","title":"가을 콘서트","venue":"올림픽홀"}

$ curl -X POST localhost:4016/concerts -d '{"title":"겨울 콘서트","venue":"KSPO DOME","settlementRate":0,"id":1}'
{"id":1,"settlementRate":0,"createdAt":"2026-10-26T04:59:11.757Z","title":"겨울 콘서트","venue":"KSPO DOME"}
```

→ 수수료율 0, id 1

© 한상훈 NestJS 백엔드 마스터

id 1번이 둘

터미널

```
$ curl localhost:4016/concerts
[{"id":1,"settlementRate":0.1,...,"title":"가을 콘서트","venue":"올림픽홀"},
 {"id":1,"settlementRate":0,...,"title":"겨울 콘서트","venue":"KSPO DOME"},
 {"id":3,"settlementRate":0.1,...,"title":12345}]

$ curl localhost:4016/concerts/1
{"id":1,"settlementRate":0.1,...,"title":"가을 콘서트","venue":"올림픽홀"}
```

→ 2번 공연: 조회 불가

© 한상훈 NestJS 백엔드 마스터

대량 할당

→ 대량 할당(mass assignment): 요청 필드를 객체에 한꺼번에 넣어, 클라이언트가 바꾸면 안 되는 필드까지 바뀌는 문제

→ 2012년 GitHub: Rails의 대량 할당으로 한 사용자가 자기 공개 키를 rails 저장소에 쓸 수 있는 키로 등록, 실제 커밋으로 공개

→ 예: 회원 가입 요청에 `"role": "ADMIN"` 한 줄 → 관리자 계정



데이터 블록: GitHub Blog "Public Key Security Vulnerability and Mitigation" (2012-03-04), OWASP API Security Top 10 2023년 버전 API3:2023 Broken Object Property Level Authorization, 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

필드를 골라 옮기기

concerts.service.ts

```
  create(createConcertDto: CreateConcertDto) {
    const concert: Concert = {
      id: this.nextId++,
      title: createConcertDto.title,
      venue: createConcertDto.venue,
      settlementRate: 0.1,
      createdAt: new Date(),
    };
    this.concerts.push(concert);
    return concert;
  }
```

○ ○ ○ 터미널

```
$ curl -X POST localhost:4016/concerts -d '{"title":"겨울 콘서트","venue":"KSPO DOME","settlementRate":0,"id":1}'
{"id":2,"title":"겨울 콘서트","venue":"KSPO DOME"}
```

© 한상훈 NestJS 백엔드 마스터

빌드된 CreateConcertDto

```ts
// create-concert.dto.ts
export class CreateConcertDto {
  title: string;
  venue: string;
}
```

```ts
// tsc 출력: create-concert.dto.js
export class CreateConcertDto {
    title;
    venue;
}
```

→ 필드 이름만 남고 타입은 사라짐

© 한상훈 NestJS 백엔드 마스터

@Body로 받은 값

```
  create(@Body() createConcertDto: CreateConcertDto) {
    console.log(createConcertDto instanceof CreateConcertDto, typeof createConcertDto.title);
```

○ ○ ○ 터미널

```
$ curl -X POST localhost:4016/concerts -d '{"title":12345}'
false number
```

→ @Body: JSON을 파싱한 일반 객체, CreateConcertDto 인스턴스 아님

© 한상훈 NestJS 백엔드 마스터

숫자 제목

터미널

```
$ curl -X POST localhost:4016/concerts -d '{"title":12345}'
{"id":3,"title":12345}
```

→ title: 숫자 그대로 저장

→ venue: 없음, 그대로 저장

→ 화면: 제목 칸에 숫자, 공연장 빈칸

→ 저장은 성공, 오류는 읽는 쪽에서

© 한상훈 NestJS 백엔드 마스터

입구의 검사

→ DTO의 타입 표시: 실행 때 검사 없음

→ 필요한 검사: 필드 타입, 필수 여부, 길이, 모르는 필드 거르기

→ 위치: 요청이 핸들러에 닿기 전

→ Nest의 도구: `ValidationPipe`

→ 지금 코드: 모르는 필드는 서비스에서 골라 옮겨 무시, 타입은 미검사

→ 서비스마다 if 검사: 메서드마다 반복, 빠뜨리기 쉬움 → 규칙은 DTO에, 검사는 입구에서

© 한상훈 NestJS 백엔드 마스터

UpdateConcertDto

update-concert.dto.ts

```ts
import { PartialType } from '@nestjs/mapped-types';
import { CreateConcertDto } from './create-concert.dto.js';

export class UpdateConcertDto extends PartialType(CreateConcertDto) {}
```

concerts.service.ts

```ts
  update(id: number, updateConcertDto: UpdateConcertDto) {
    const concert = this.findOne(id);
    if (updateConcertDto.title !== undefined) concert.title = updateConcertDto.title;
    if (updateConcertDto.venue !== undefined) concert.venue = updateConcertDto.venue;
    return concert;
  }
```

→ `PartialType`: CreateConcertDto의 필드를 전부 선택 필드로

© 한상훈 NestJS 백엔드 마스터

PATCH에 끼워 넣은 수수료율

터미널

```
$ curl -X PATCH localhost:4016/concerts/1 -d '{"venue":"KSPO DOME","settlementRate":0.5}'
{"id":1,"title":"가을 콘서트","venue":"KSPO DOME"}
```

→ 바뀐 것: venue

→ `settlementRate`: update가 고르지 않아 그대로 0.1

© 한상훈 NestJS 백엔드 마스터

엔티티가 테이블이 되면

바뀌는 것	엔티티를 그대로 응답	응답 DTO
컬럼 이름 `venue` → `venue_name`	응답 필드 이름 변경, 앱 화면 깨짐	`from()`에서 한 줄 수정, 응답 그대로
내부 컬럼 추가	응답에 자동 노출	영향 없음
테이블 관계 추가	연결된 데이터까지 응답에 섞일 수 있음	고른 필드만

→ 이미 설치된 옛 버전 앱: 바뀐 응답을 계속 받음

→ Date 필드: JSON에서 ISO 문자열(예: `2026-10-26T04:59:11.724Z`), 형식은 응답 DTO에서 결정

© 한상훈 NestJS 백엔드 마스터

세 클래스의 경계

클래스	적는 것	누가 보나
`CreateConcertDto`	클라이언트가 보내도 되는 필드	요청을 보내는 쪽
`Concert`	서버가 관리하는 필드 전부	서비스들
`ConcertResponseDto`	보여 줘도 되는 필드	응답을 받는 쪽

→ 필드를 붙일 때: 클라이언트가 정해도 되나 → 요청 DTO / 봐도 되나 → 응답 DTO / 둘 다 아니면 엔티티만

© 한상훈 NestJS 백엔드 마스터

회차와 좌석의 응답 DTO

seat-response.dto.ts

```ts
export class SeatResponseDto {
  id: string;
  label: string;
  status: SeatStatus;

  static from(seat: Seat): SeatResponseDto {
    return { id: seat.id, label: seatLabel(seat.zone, seat.row, seat.no), status: seat.status };
  }
}
```

○ ○ ○ 터미널

```
$ curl -X POST localhost:4016/performances/1/seats/A-3-15/hold
{"id":"A-3-15","label":"A구역 3열 15번","status":"HELD"}
```

→ 회차: `PerformanceResponseDto` (id, concertId, startsAt)

→ 좌석: 엔티티에 없는 label을 응답에서 만듦 (libs/common의 seatLabel)

© 한상훈 NestJS 백엔드 마스터

지금의 공연 모듈 파일

```
concerts/
├── concerts.module.ts          exports: ConcertsService
├── concerts.controller.ts      응답 DTO로 변환, ParseIntPipe, 204
├── concerts.service.ts         필드를 골라 엔티티 생성, 없으면 404
├── dto/
│   ├── create-concert.dto.ts     title, venue
│   ├── update-concert.dto.ts     PartialType
│   └── concert-response.dto.ts   id, title, venue
└── entities/
    └── concert.entity.ts       + settlementRate, createdAt
```

© 한상훈 NestJS 백엔드 마스터

DTO와 엔티티의 규칙

"요청 DTO에는 받을 것만, 응답 DTO에는 보여 줄 것만 적고,
엔티티는 밖으로 그대로 내보내지 않습니다."